

✓ Exercício 1 - Referências

Responda:

- Qual será a saída?
- a e b representam o mesmo objeto?
- Por quê?
- O que aconteceria com `b = a.copy()`?

```
1 a = [10, 20, 30]
2 b = a.copy()
3
4 a.append(40)
5
6 print("a:", a)
7 print("b:", b)
8 print("id(a):", id(a))
9 print("id(b):", id(b))
```

```
a: [10, 20, 30, 40]
b: [10, 20, 30, 40]
id(a): 132769305814016
id(b): 132769305809216
```

A saída foi:

a: [10, 20, 30, 40]

b: [10, 20, 30, 40]

id(a): 132769305809984

id(b): 132769305809984

Os valores de A foram declarados e B foi declarado como A, sendo assim, eles representam o mesmo objeto

"`b = a.copy()`" faz uma cópia de A da maneira que ele encontra naquele momento e não acompanha mudanças.

✓ Exercício 2 — Construindo uma cadeia

```
1 class Node:
2     def __init__(self, valor):
3         self.valor = valor
4         self.proximo = None
5
6 n1 = Node("A")
7 n2 = Node("B")
8 n3 = Node("C")
```

Conecte:

$A \rightarrow B \rightarrow C \rightarrow \text{None}$

Depois percorra a estrutura e produza:

A

B

C

✓ Exercício 3 — Depuração

```
1 class Node:
2     def __init__(self, valor):
3         self.valor = valor
4         self.proximo = None
5
6 n1 = Node(10)
7 n2 = Node(20)
```

```

8 n3 = Node(30)
9
10 n1.proximo = n2
11 n2.proximo = n3
12
13 atual = n1
14
15 while atual is not None:
16     print(atual.valor)
17     atual = atual.proximo

```

```

10
20
30

```

Perguntas:

Qual é o problema? O problema é que a variável atual nunca é atualizada dentro do while. Ela continua apontando para o primeiro nó (n1).

Por que o programa não termina? Porque atual continua sendo n1, que não é None. Assim, a condição atual is not None permanece verdadeira e o programa entra em um loop infinito, imprimindo 10 repetidamente.

Qual linha deve ser acrescentada? Dentro do while, depois do print, deve ser acrescentada: atual = atual.proximo

Qual será a saída depois da correção? 10, 20, 30

✓ 18. Problema real — Sistema de atendimento de uma clínica

```

1 class Paciente:
2     def __init__(self, nome, idade, prioridade):
3         self.nome = nome
4         self.idade = idade
5         self.prioridade = prioridade
6
7
8 class Node:
9     def __init__(self, paciente):
10        self.paciente = paciente
11        self.proximo = None
12
13
14 class FilaAtendimento:
15     def __init__(self):
16         self.inicio = None
17         self.fim = None
18         self.quantidade = 0
19
20     def adicionar(self, paciente):
21         novo = Node(paciente)
22
23         # Se a fila estiver vazia
24         if self.inicio is None:
25             self.inicio = novo
26             self.fim = novo
27
28         # Se o paciente for prioritário, entra no início
29         elif paciente.prioridade.lower() == "prioridade":
30             novo.proximo = self.inicio
31             self.inicio = novo
32
33         # Paciente normal entra no final
34         else:
35             self.fim.proximo = novo
36             self.fim = novo
37
38         self.quantidade += 1
39
40     def atender(self):
41         if self.esta_vazia():
42             return None
43
44         paciente = self.inicio.paciente
45         self.inicio = self.inicio.proximo
46
47         # Se a fila ficou vazia
48         if self.inicio is None:
49             self.fim = None
50
51         self.quantidade -= 1

```

```

52
53     return paciente
54
55     def listar(self):
56         if self.esta_vazia():
57             print("A fila está vazia.")
58             return
59
60         atual = self.inicio
61
62         while atual is not None:
63             paciente = atual.paciente
64
65             print(
66                 paciente.nome,
67                 "-",
68                 paciente.idade,
69                 "anos -",
70                 paciente.prioridade
71             )
72
73             atual = atual.proximo
74
75     def esta_vazia(self):
76         return self.inicio is None
77
78     def tamanho(self):
79         return self.quantidade
80
81
82 # Criando a fila
83 fila = FilaAtendimento()
84
85 # Adicionando pacientes
86 fila.adicionar(Paciente("Ana", 32, "Normal"))
87 fila.adicionar(Paciente("Bruno", 70, "Normal"))
88 fila.adicionar(Paciente("Carlos", 45, "Prioridade"))
89 fila.adicionar(Paciente("Daniela", 28, "Normal"))
90 fila.adicionar(Paciente("Eduardo", 65, "Prioridade"))
91
92 # Listando a fila
93 print("Fila de atendimento:")
94 fila.listar()
95
96 print("\nQuantidade de pacientes:", fila.tamanho())
97
98 # Atendendo pacientes
99 paciente = fila.atender()
100
101 if paciente is not None:
102     print("\nAtendendo:", paciente.nome)
103
104 print("\nFila após o atendimento:")
105 fila.listar()
106
107 print("\nQuantidade de pacientes:", fila.tamanho())

```

Fila de atendimento:
 Eduardo - 65 anos - Prioridade
 Carlos - 45 anos - Prioridade
 Ana - 32 anos - Normal
 Bruno - 70 anos - Normal
 Daniela - 28 anos - Normal

Quantidade de pacientes: 5

Atendendo: Eduardo

Fila após o atendimento:
 Carlos - 45 anos - Prioridade
 Ana - 32 anos - Normal
 Bruno - 70 anos - Normal
 Daniela - 28 anos - Normal

Quantidade de pacientes: 4

